

Recommender Systems

Diploma Thesis

Supervisor:

Prof. Ing. Radim Farana, CSc., Feng.

Apoorv Kala

Brno 2024

Statutory declaration

Herewith I declare that I have written my final thesis: Recommender Systems by myself and all sources and data used are quoted in the list of references. I agree that my work will be published in accordance with Section 47b of Act No. 111/1998 Sb. On Higher Education as amended thereafter and in accordance with the *Guidelines on the Publishing of University Student Theses*.

I am aware of the fact that my thesis is subject to Act. No. 121/2000 Sb., the Copyright Act and that the Mendel University in Brno is entitled to close a licence agreement and use the results of my thesis as the "School Work" under the terms of Section 60 para. 1 of the Copyright Act.

Before closing a licence agreement on the use of my thesis with another person (subject) I undertake to request for a written statement of the university that the licence agreement in question is not in conflict with the legitimate interests of the university, and undertake to pay any contribution, if eligible, to the costs associated with the creation of the thesis, up to their actual amount.

In Brno on 10th May 2024

Contents

1	Introduction	13
1.1	Background and motivation	13
1.2	Research Questions.....	13
1.3	Objectives of the study	14
1.4	Structure of thesis.....	14
2	Literature Review	16
2.1	Overview	16
2.2	Traditional Recommender System	16
2.2.1	Collaborative Filtering.....	16
2.2.2	Types of Collaborative Filtering	17
2.2.3	Memory-based Collaborative Filtering	18
2.2.4	Model-based Collaborative Filtering	18
2.2.5	Strengths of Collaborative Filtering	19
2.2.6	Weaknesses of Collaborative Filtering	19
2.2.7	Examples of Collaborative Filtering in Recommender System	19
2.3	Content-Based Filtering	20
2.3.1	Strengths of Content-Based Filtering	21
2.3.2	Challenges of Content-Based Filtering.....	22
2.3.3	Examples of Content-Based Filtering in Recommender Systems	22
2.4	Hybrid Approaches	22
2.4.1	Examples of Hybrid Filtering in Recommender Systems	23
2.5	Fuzzy Logic in Recommender Systems	23
2.5.1	Fuzzy Logic membership function	26
2.5.2	Application of Fuzzy Logic	26
2.5.3	Fuzzy Rules.....	27
2.5.4	Defuzzification.....	27
2.5.5	Difference between Fuzzification and Defuzzification.....	28

2.6	Evaluation Metrics for Recommender Systems	28
2.6.1	Accuracy	28
2.6.2	F1 Score	29
2.6.3	Precision.....	29
2.6.4	Recall.....	30
2.6.5	Confusion Matrix	30
2.6.6	Evaluation of Presented Research Questions	31
3	Methodology	33
3.1	Data Collection	34
3.2	Development of the fuzzy logic expert system	35
3.2.1	Problem Statement	35
3.2.2	Variables and Linguistic terms	35
3.2.2.1	Input Variables	35
3.2.2.2	Output Variables.....	36
3.2.3	Fuzzy logic rules	36
3.2.4	Membership Functions	37
3.2.5	Fuzzy Inference Engine	38
3.2.6	Knowledge Base.....	39
3.2.7	Inference Engine Implementation	39
3.2.8	User Interface Integration	40
3.2.9	Testing and Validation.....	41
3.2.10	Deployment and Monitoring	42
3.3	Development of Traditional Recommender Systems	43
4	Results	45
4.1	Data Preprocessing	45
4.2	Fuzzy Logic	47
4.3	Content Based ML Model	49
4.4	Evaluation	51
4.4.1	Fuzzy Logic.....	51
4.4.2	Content-Based ML model	53

5	Discussion	55
6	Conclusion	56
7	References	57
8	List of figures	59
9	List of tables	60

1 Introduction

1.1 Background and motivation

In the contemporary digital landscape, the overwhelming volume of information poses a challenge for users to discover relevant content. Recommender systems have emerged as a pivotal solution, leveraging algorithms to predict user preferences and provide personalized recommendations.

Traditional recommender systems, often grounded in collaborative filtering or content-based approaches, have demonstrated effectiveness in various domains. However, the evolving nature of user preferences and the need for more nuanced decision-making call for innovative approaches.

Motivating this research is the quest to enhance the precision and interpretability of recommender systems through the integration of fuzzy logic into expert systems. Fuzzy logic, with its ability to handle imprecise information and uncertainty, presents a promising avenue for refining the recommendation process.

This study aims to investigate the amalgamation of fuzzy-logic principles with expert systems to develop a novel recommender system, offering potentially more adaptable and context-aware recommendations.

1.2 Research Questions

This study seeks to address the following research questions:

1. What are the underlying principles and methodologies of traditional recommender systems, and how do they contribute to user engagement?
2. How can fuzzy logic be effectively incorporated into an expert system to improve the interpretability and adaptability of recommendations?
3. To what extent does a fuzzy-logic-based expert system outperform traditional recommender systems in terms of accuracy, relevance, and user satisfaction?
4. What are the comparative strengths and weaknesses of the fuzzy-logic expert system and traditional recommender systems across different scenarios and user segments?

1.3 Objectives of the study

The overarching objectives of this master's thesis are:

1. To comprehensively review existing literature on recommender systems, fuzzy logic, and their amalgamation.
2. To develop a fuzzy-logic-based expert system for personalized recommendations.
3. To implement a traditional recommender system for comparative analysis.
4. To rigorously evaluate the performance of both systems, considering metrics of accuracy, relevance, and user satisfaction.
5. To compare the strengths and weaknesses of the fuzzy-logic expert system with traditional recommender systems.
6. To explore user preferences, trust factors, and the overall experience associated with recommendations from both systems.
7. To derive generalized findings applicable across diverse domains and provide recommendations for the advancement of recommender systems.

1.4 Structure of thesis

The remainder of this thesis is organized as follows:

- Chapter 2: Literature Review provides a comprehensive overview of recommender systems, fuzzy logic, and related methodologies.
- Chapter 3: Methodology details the research design, data collection, and the development of the fuzzy-logic expert system and traditional recommender system.
- Chapter 4: Results presents the findings of the evaluation and comparison of both systems.

-
- Chapter 5: Discussion interprets the results, discusses implications, limitations, and proposes future research directions.
 - Chapter 6: Conclusion summarizes the study, its contributions, and outlines recommendations for practitioners and researchers.

The subsequent chapters delve into each aspect of the study, culminating in a holistic exploration of the interplay between fuzzy logic and recommender systems.

2 Literature Review

2.1 Overview

(Rocca, 2019) Over the past few decades, the increasing prominence of platforms like YouTube, Amazon, and Netflix, among others, has led to the widespread integration of recommender systems into our daily lives. These systems play a crucial role in various domains, from suggesting relevant products to potential buyers in e-commerce to matching user preferences with appropriate content in online advertising. As a result, recommender systems have become an integral part of our online experiences.

At their core, recommender systems are algorithms designed to recommend suitable items to users, such as movies, articles, products, or other relevant offerings based on industry-specific requirements. These systems hold significant importance in certain industries, as their effectiveness can generate substantial revenue and provide a competitive edge over rivals. An illustration of their significance is Netflix's past initiative, the "Netflix prize," where they offered a prize of 1 million dollars to anyone who could develop a recommender system that outperformed their own algorithm.

This thesis aims to explore different paradigms of recommender systems, examining their operational mechanisms, theoretical foundations, and evaluating their strengths and weaknesses.

2.2 Traditional Recommender System

2.2.1 Collaborative Filtering

A set of items that are dependent on the user's previous selections is necessary for collaborative filtering. For this system to function, not many product features are necessary. Each item and user are described by an embedding or feature vector, which sinks them both in the same embedding place. It independently generates enclosures for both users and objects. When recommending a specific product to the main user, the responses of other buyers are considered. Before proposing the item that customers like the most, it takes note of how each user behaves. When recommending a product to the main consumer, it also connects similar users based on similarities in preferences and behavior towards a related product. The most well-known application suggestion engine, Collaborative Filtering, makes intelligent predictions about which users would like a given product in the future. This type of algorithm is also known as a product-based collaborative shift. Instead of things, in this filtering, users are screened and linked to each User. Only users' behavior is considered in this system. Only their profile information and content are insufficient. Users who rate products favorably will be linked to other users who act in a similarly favorable manner.

2.2.2 Types of Collaborative Filtering

There are two types of Collaborative Filtering process:

1. Memory-based Collaborative Filtering
2. Model-based Collaborative Filtering

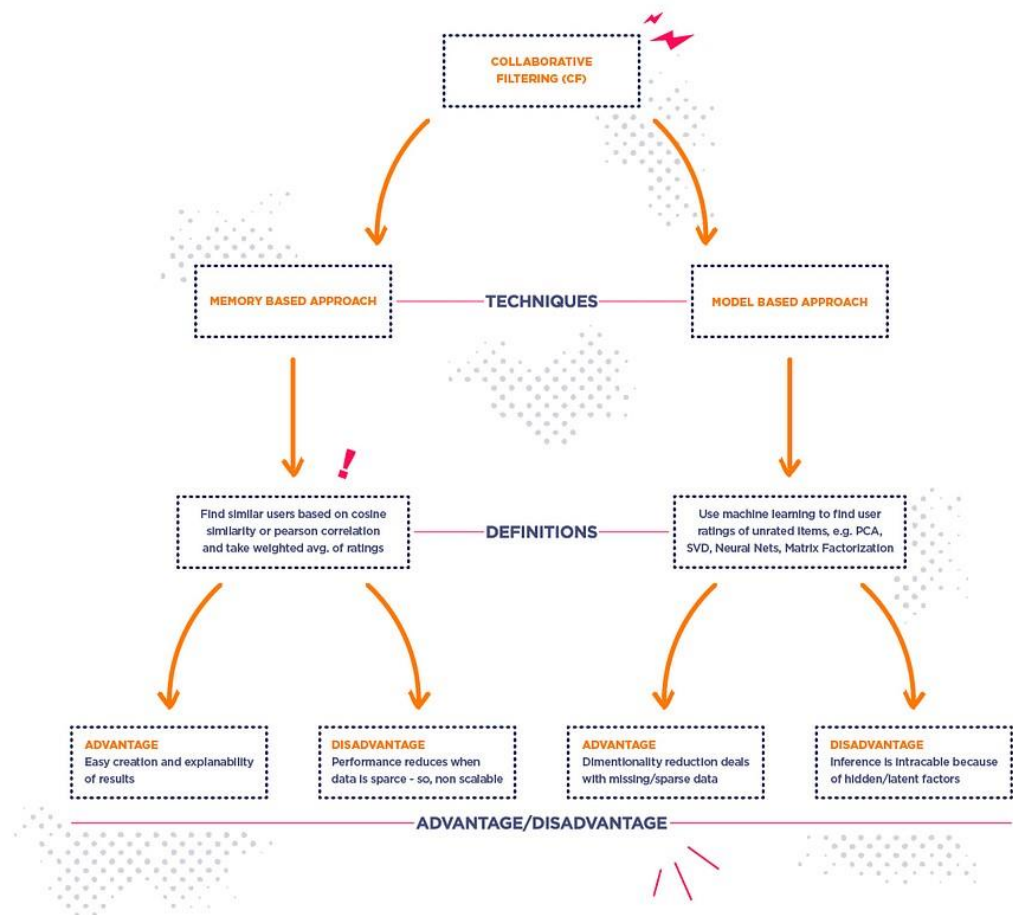


Figure 1 - Collaborative Filtering (Source: www.iteratorshq.com)

2.2.3 Memory-based Collaborative Filtering

(Collaborative Filtering - Business Intelligence | WYgroup BI, 2020) Collaborative filtering (CF) is a technique used by recommender systems. Collaborative filtering has two senses, a narrow one and a more general one.

Collaborative filtering, in its more recent, condensed form, is a technique for making predictions (or filtering) about a user's interests automatically while simultaneously gathering preferences or taste data from many users. The collaborative filtering method's core premise is that if two people have the same view on a subject, they are more likely to have it on a different subject than two randomly selected people. For instance, given a partial list of a user's preferences (likes or dislikes), a collaborative filtering recommendation system for television programs could make predictions about which television show the user should like. It should be noted that although these forecasts are based on data from numerous users, they are specific to the individual.

This is distinct from the more straightforward method of assigning an average (non-specific) score for each interesting item, for instance based on the quantity of votes it has received.

In a broader sense, collaborative filtering refers to the method of searching for patterns or information by working with a variety of agents, viewpoints, data sources, etc. Collaborative filtering applications sometimes include enormous data sets. Numerous types of data have been subjected to collaborative filtering techniques, such as sensing and monitoring data used in mineral exploration, environmental sensing over wide areas, or data collected by numerous sensors; financial data used by financial service providers who integrate numerous financial sources; or data used in e-commerce and web applications where the focus is on user data, etc. The rest of the discussion will concentrate on collaborative filtering for user data, however some of the techniques and strategies may also be applicable to the other principal applications.

2.2.4 Model-based Collaborative Filtering

Model-based collaborative filtering provides recommendations by developing a model from user ratings. In addition to using explicit data such as ratings, collaborative filtering can also use implicit information by observing the habits of users, such as music played, applications downloaded,

Websites visited, or books read. To develop a model, there are two approaches that can be used, which are probability approach or rating prediction. The modeling process is conducted by machine learning techniques such as classification, clustering, and rule-based approach. Based on its characteristics, model-based also has its advantages and disadvantages. Su and Khoshgoftaar stated that modelbased approach has better predictions than memory based. It is also capable of handling the problem of sparsity and scalability better than memory based. However, model-based

approach requires a great resource, such as time and memory, to develop the model and may lose information when using dimensionality reduction.

2.2.5 Strengths of Collaborative Filtering

- **Serendipitous Recommendations:** Collaborative filtering can provide serendipitous recommendations by identifying items that users with similar preferences have liked or rated highly. It can introduce users to new and unexpected items they might not have discovered otherwise.
- **Scalability:** Collaborative filtering can handle large-scale datasets efficiently. The computation involved in identifying similar users or items can be distributed or parallelized, making it suitable for systems with many users and items.
- **Cold-Start Problem:** Collaborative filtering can handle the cold-start problem, where there is limited or no information about new users or items. By leveraging the preferences of similar users or items, collaborative filtering can make meaningful recommendations even in the absence of user or item data.

2.2.6 Weaknesses of Collaborative Filtering

- **Data Sparsity:** Collaborative filtering relies on user-item interaction data, and in many cases, such data can be sparse. When users have limited interactions or when the number of items is large, it can be challenging to find sufficient overlap between users' preferences, leading to inaccurate or less diverse recommendations.
- **Scalability with Growing User Base:** As the number of users grows, the computation required to identify similar users or items increases, impacting the scalability of collaborative filtering. The computational complexity can become a challenge as the user base expands.
- **Cold-Start Problem for New Users or Items:** While collaborative filtering can handle the cold-start problem to some extent, it still faces challenges when dealing with entirely new users or items for which no or minimal data is available. It can be difficult to provide accurate recommendations without sufficient historical data.

2.2.7 Examples of Collaborative Filtering in Recommender System

2.2.7.1 User-Based Collaborative Filtering

In this approach, recommendations are made based on the similarities between users. The system identifies users with similar preferences and recommends items that those similar users have liked or rated highly.

For example, if User A and User B have similar tastes and preferences, and User A has rated Item X highly, the system would recommend Item X to User B.

2.2.7.2 Item-Based Collaborative Filtering

This approach focuses on the similarities between items rather than users. It recommends items based on the associations between items and the user's past interactions.

For example, if User A has shown interest in and interacted with Item X, and Item X is like Item Y based on their attributes or previous user interactions, the system would recommend Item Y to User A.

2.2.7.3 Matrix Factorization

Matrix factorization techniques, such as Singular Value Decomposition (SVD) and Alternating Least Squares (ALS), are commonly used in collaborative filtering. These techniques decompose the user-item interaction matrix into latent factors that capture underlying preferences and relationships between users and items. Recommendations are then generated by predicting missing values in the matrix or estimating user preferences based on the learned latent factors.

2.2.7.4 Neighborhood-Based Collaborative Filtering

This approach identifies neighborhoods of similar users or items and uses their preferences to make recommendations. User-based neighborhood collaborative filtering looks for users with similar tastes and recommends items that their neighbors have liked. Item-based neighborhood collaborative filtering identifies items that are similar based on user interactions and recommends items that are highly associated with the user's previous preferences.

2.3 Content-Based Filtering

(Roy, 2020) A content-based recommender system seeks to infer a user's characteristics or behavior based on the qualities of the item to which the user responds favorably.

Movies	User 1	User 2	User 3	User 4	Action	Comedy
Item 1	1		4	5	Yes	No
Item 2	5	4	1	2	No	Yes
Item 3	4	4		3	Yes	Yes
Item 4	2	2	4	4	No	Yes

Table 1 - Movie Recommendation

The final two columns, Action, and Comedy, list the film genres. Given these genres, we can now identify which users prefer which genre, allowing us to develop features tailored to that user based on how they respond to films in that genre. Once we are aware of the user's preferences, we can use the feature vector that was produced to embed the user in an embedding space and provide recommendations based on their preferences. The user's favorite feature vectors from prior records are combined with the item's feature vectors to generate the similarity metrics (we'll talk about it in a moment). The best few are then suggested.

When making suggestions to one person, content-based filtering does not need the data of other users.

2.3.1 Strengths of Content-Based Filtering

- At present time, no user data is required for the creation of suggestions. Content-based filtering does not require user data to generate recommendations, in contrast to collaborative filtering. A content-based filtering system may start recommending pertinent information after a user has searched for, browsed through, and/or purchased a few items. Therefore, it is perfect for businesses without a sizable user base to test it on. It also works effectively for retailers in certain product categories or specialized markets where there is a significant user base but little user interaction.
- Recommendations will be very useful to the user. Content-based recommenders can be highly tailored to the user's tastes, including recommendations for specific products, because the process depends on matching the attributes of a database object with the user's profile. For example, content-based filtering may be able to recognize a specific user's preferences and interests. Content-based filtering is extremely beneficial for companies that have enormous product libraries of a single type, like those that manufacture phones, where suggestions must be based on a range of different features.
- You solve the "cold start" problem. Collaborative filtering might result in a potential cold start issue when a new website or community has few brandnew users and few user connections. Although content-based filtering requires some initial input

from users to begin providing recommendations, the quality of early recommendations is typically better than those made by a collaborative system, which must accumulate and correlate millions of data points before becoming optimized.

2.3.2 Challenges of Content-Based Filtering

- There is a lack of both variation and novelty. Recommendations consider more factors than merely relevancy. Think about how much you liked the movie *Tenet*. Without a sure, you'll enjoy *Inside Man* as well. It's possible that you already know this; in that case, a recommender system won't be necessary. As a result, recommendation engines need to produce a variety of surprising results to be effective.
- Scalability is difficult. Each new product, service, or piece of knowledge must have its features documented and categorized. Scalability can be challenging and time-consuming since attribute assignments are laborious and never-ending.
- The traits could be unreliable or inconsistent. The quality of suggestions based on content is determined by the level of competence of the experts who categorize items. Many of the millions of things that require qualities may be misclassified since they may be arbitrary. There must be a system in place to guarantee that attributes are applied accurately and consistently. A content-based recommender system won't work properly otherwise.

2.3.3 Examples of Content-Based Filtering in Recommender Systems

1. eCommerce Recommendations
2. Media Recommendations
3. Video Games and Stores
4. Location-Based Recommendations

2.4 Hybrid Approaches

(B. Thorat et al., 2015) Hybrid filtering combines several filtering strategies. The cold start problem, overspecialization problem, and sparsity problem are only a few of the prevalent issues with the filtering approaches that the hybrid filtering methodology has been created to address.

The use of hybrid filtering is influenced by the aim to increase the precision and effectiveness of the recommendation process. In essence, the goal of a hybrid approach is to combine collaborative filtering and content-based filtering to increase the accuracy of recommendations. There are several ways to implement hybrid approaches:

-
- Individually apply collaborative and content-based techniques, then combine their forecasts.
 - Create a broad consolidative model that incorporates both content-based and collaborative characteristics.
 - Integrate certain content-based characteristics into a collaborative approach.
 - Comprise some collaborative characteristics into a content-based approach.

2.4.1 Examples of Hybrid Filtering in Recommender Systems

Hybrid recommender systems combine multiple recommendation techniques to leverage their strengths and provide more accurate and diverse recommendations.

Here are a few examples of hybrid recommender systems:

1. **Weighted Hybrid:** In this approach, recommendations from different techniques (e.g., collaborative filtering and content-based filtering) are combined using weighted averages or scores.
2. **Switching Hybrid:** In this approach, recommendations from different techniques (e.g., collaborative filtering and content-based filtering) are combined using weighted averages or scores.
3. **Feature Combination:** In this approach, features extracted from both collaborative and content-based techniques are combined to enhance recommendation accuracy.
4. **Cascade Hybrid:** This approach involves using one technique to pre-filter the item set and then applying another technique on the pre-filtered set.
5. **Meta-level Hybrid:** In this approach, the recommendations from different techniques are treated as "meta-level" information and used to train a higher-level model. The meta-level model learns to combine the recommendations from various techniques and generate a final recommendation.

2.5 Fuzzy Logic in Recommender Systems

(Gupta & Jain, 2018) A recommender system considers both the user's preferences and the features of the item. These criteria and their representation affect how well a recommender system performs and operates. These two elements are essential in showing a user how personalized a service is.

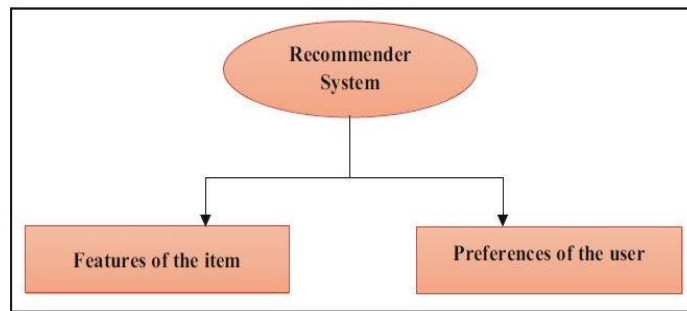


Figure 2 – Fuzzy Logic in Recommender Systems

However, these variables are arbitrary and ill-defined. The recommender system will have a difficult time managing the uncertainty as a result. It's also noteworthy to note that these uncertainties affect the goods' attributes to varying degrees (low to high). A car might, for instance, have interiors that are very good, okay, not so good, horrible, or awful.

To handle this ambiguity, fuzzy logic in recommender systems is highly advised. In addition to assisting in the logical modeling of vagueness, this offers heuristic recommendations and a cogent framework for recommender system design. In literature, a recommender system has been developed utilizing fuzzy logic. Due to the use of fuzzy logic, the following substantial changes have been made to the recommendation:

1. The fuzzy rules help to model the vagueness in user input in an efficient manner with the help of linguistic variables ("like"/"dislike"/"awful"/"good").
2. Use of fuzzy logic with other techniques like clustering improves the determination of similar users, by finding the fuzzy similarity degree between two users,
3. The closeness between two fuzzy ratings (user input) over a particular product has been understood in an easy way with the help of fuzzy rules, (Mbaabu, 2020) Fuzzy logic is based on a simple idea - a membership function. This function tells you how closely a thing belongs to a certain group. This function gives a number between 0 and 1 to show how much someone or something belongs to a group. If the number is 0, it means they don't belong at all. If it's 1, it means they fully belong.

Using fuzzy logic involves using fuzzy rules. These rules are like sentences that explain how the input and output variables are connected, but in an unclear way. Fuzzy logic results in a fuzzy set that contains degrees of likelihood for different outputs. In the Boolean system, 1 is true and 0 is false.

The fuzzy system doesn't have a way to determine if something is completely true or false. Fuzzy logic allows for something to be partly true and partly false.

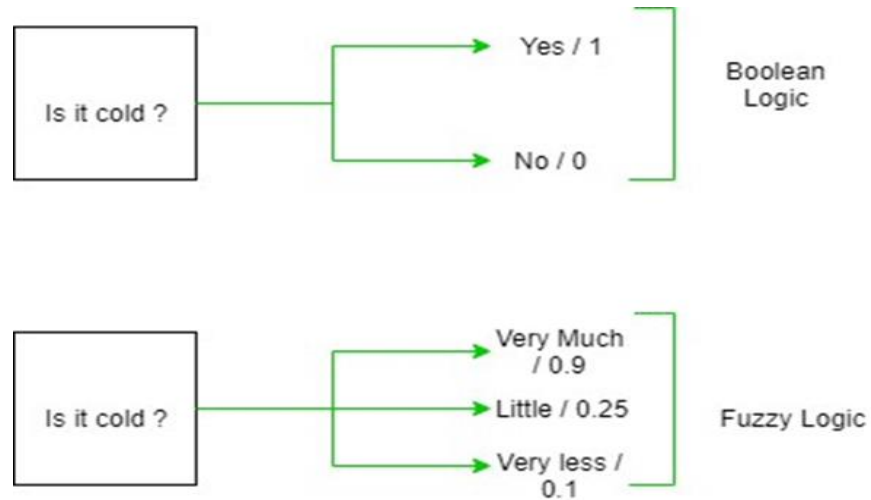


Figure 3 – Fuzzy Logic (<https://www.geeksforgeeks.org/fuzzy-logic-introduction/>)

Fuzzy logic architecture consists of the following components:

- **Rule Base:** This is what tells the fuzzy logic system how to make decisions using certain rules and membership functions. It also has rules for when things should happen and how to control the system.
- **The fuzzifier** changes basic inputs into fuzzy groups. After being introduced to the control system, the fuzzy sets undergo further modifications.
- **The Inference Engine** figures out the best rules based on what you give it. After that, it uses these rules to make a fuzzy result from the data given.
- **The Defuzzifier** changes fuzzy sets into clear outputs that are easy to understand. Defuzzification is the last step in a fuzzy logic system.

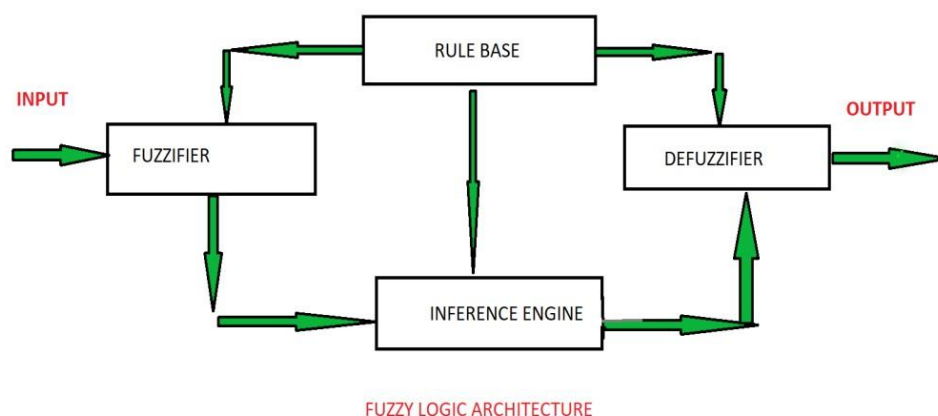


Figure 4 – Fuzzy Logic architecture (<https://www.geeksforgeeks.org/fuzzylogic-introduction/>)

2.5.1 Fuzzy Logic membership function

A membership function is a graphical representation of a fuzzy set. It shows how values ranging between 0 and 1 are mapped to inputs. Inputs are generally represented as Universe (U). The membership function for a given fuzzy set is in the form: $\mu_A: X \rightarrow [0,1]$

Where A is a fuzzy set and X is the Universe.

Any value within the range of 0 to 1 indicates a degree of membership. Each element of the Universe (X) is given a specific degree of membership.

2.5.2 Application of Fuzzy Logic

(Sayantini, 2023) Fuzzy logic has applications in a variety of industries, including those that govern the environment, residential appliances, and automobile systems. Common uses include:

- Controlling the height of satellites and spacecraft in the aerospace industry.
- It is also used to regulate the flow of traffic and speed in the automobile systems.
- It is utilized in huge corporate business for personal evaluation and decision assistance systems.
- In the chemical sector, it is also used to regulate the pH, drying, and chemical distillation processes.

- Fuzzy logic is employed in many sophisticated applications of artificial intelligence, including natural language processing.
- Expert systems, one type of contemporary control system, make heavy use of it.
- Fuzzy logic mirrors human decision-making, but much more quickly. As a result, it is compatible with neural networks.

2.5.3 Fuzzy Rules

In general, the logical inference is based on the evaluation of decision rules in the form of conditional statements.

IF THEN....

For inferring knowledge, the condition is expressed in the form of an implication of two fuzzy statements mostly as:

IF <fuzzy expression> THEN <fuzzy expression>

This condition is referred to as the "fuzzy IF-THEN rule". The first fuzzy expression set, which is often a compound statement, is called the antecedent, where individual parts of the statement are bound by logical conjunctions. The second fuzzy statement is consequential.

2.5.4 Defuzzification

Defuzzification is a vital process in recommender systems that aims to transform fuzzy logic-based recommendations into actionable and understandable outputs (Jain & Gupta, 2018). In the realm of recommendation engines, where user preferences and item characteristics often exhibit uncertainty and ambiguity, defuzzification plays a pivotal role in converting fuzzy recommendations into crisp, numerical values or rankings.

This conversion facilitates effective decision-making and enhances user experience by providing clear and concise recommendations. Through defuzzification techniques such as the centroid method, weighted average method, max membership method, and bisector method, recommender systems are able to present users with concrete recommendations that reflect the underlying fuzzy logic while maintaining clarity and usability.

A fuzzy set is mapped to a crisp set using this procedure. Usually, fuzzy control systems require it. These systems will contain several rules that combine a variety of factors into a result that is fuzzy, or one that can be defined in terms of membership in fuzzy sets. Rules that determine how much pressure to apply, for instance, might produce the following results: "Decrease Pressure (15%), Maintain Pressure (34%), Increase Pressure (72%)". Defuzzification is the process of translating a fuzzy set's membership degrees into a concrete choice or real value.

2.5.5 Difference between Fuzzification and Defuzzification

S.No.	Comparison	Fuzzification	Defuzzification
1.	Basic	Precise data is converted into imprecise data.	Imprecise data is converted into precise data.
2.	Definition	Fuzzification is the method of converting a crisp quantity into a fuzzy quantity.	Defuzzification is the inverse process of fuzzification where the mapping is done to convert the fuzzy results into crisp results.
3.	Example	Voltmeter	Stepper motor and D/A converter
4.	Methods	Intuition, inference, rank ordering, angular fuzzy sets, neural network, etcetera.	Maximum membership principle, centroid method, weighted average method, center of sums, etcetera.
5.	Complexity	It is quite simple.	It is quite complicated.
6.	Use	It can use IF-THEN rules for fuzzifying the crisp value.	It uses the center of gravity methods to find the centroid of the sets.

Table 2 - Fuzzification and Defuzzification ((Katiyar, 2022))

2.6 Evaluation Metrics for Recommender Systems

2.6.1 Accuracy

Definition: Accuracy is a measure of the overall correctness of the model. It calculates the ratio of correctly predicted instances to the total instances.

(Tigerschiold, 2022) Accuracy is the most common metric to be used in everyday talk. Accuracy answers the question “Out of all the predictions we made, how many were true?”

Formula:

$$accuracy = \frac{true\ positives + true\ negatives}{true\ positives + true\ negatives + false\ negatives + false\ positives}$$

Use Case: It is a useful metric when the classes in the dataset are balanced. However, it might not be appropriate in cases where the classes are imbalanced.

2.6.2 F1 Score

Definition: (Tigerschiold, 2022) F1 Score is a measure that combines recall and precision. As we have seen there is a trade-off between precision and recall, F1 can therefore be used to measure how effectively our models make that trade-off. F1 Score is the harmonic mean of precision and recall. It provides a balance.

Formula:

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

Use case: Particularly useful when there is an uneven class distribution and both false positives and false negatives are critical.

2.6.3 Precision

Precision is a crucial metric for evaluating the performance of recommender systems. It measures the accuracy of the recommendations made by the system by assessing how many of the recommended items are relevant to the user's preferences or needs. Precision is calculated by dividing the number of relevant items recommended by the system by the total number of items recommended. However, it's important to define what constitutes a "relevant" item. This definition can vary depending on the context and the goals of the recommendation system. For example, in a movie recommendation system, a relevant item could be a movie that the user enjoys or rates highly.

$$Precision = \frac{\text{Number of relevant items recommended}}{\text{Total number of items recommended}} \times 100\%$$

High precision indicates that the recommender system is effectively recommending items that are relevant to the user, while low precision suggests that the recommendations are less accurate or helpful. Precision is particularly important in scenarios where the user's attention is limited, such as when presenting a small list of recommendations or when recommending expensive or time-consuming items. In such cases, it's crucial to ensure that the recommended items are highly relevant to the user's interests or needs to maximize user satisfaction and engagement.

However, precision should be considered alongside other metrics such as recall, which measures the system's ability to recommend all relevant items, and accuracy, which evaluates the overall correctness of the recommendations. Balancing these metrics can help build a more effective and reliable recommender system that meets the diverse needs of users.

2.6.4 Recall

Recall stands out as a pivotal measure in the realm of assessing recommender systems through evaluation metrics. It gauges the fraction of pertinent items effectively suggested by the system. As an accuracy metric, it offers insights into the system's proficiency in surfacing items aligning with a user's preferences.

$$\text{Recall} = \frac{\text{Number of relevant items recommended}}{\text{Total number of items recommended}}$$

The fundamental question posed by the concept of recall in recommender systems is how many of the items that a user deems relevant were accurately recommended by the system. A higher recall signifies the system's adeptness in capturing a substantial portion of relevant items, while a lower recall hints at potential gaps where relevant items may be overlooked or not suggested.

2.6.5 Confusion Matrix

The confusion matrix, although traditionally associated with classification tasks, can be adapted and utilized effectively in evaluating recommendation systems. Let's delve into its utility through a literature review. Several studies have explored the applicability of confusion matrices in the context of recommendation systems. These studies emphasize the importance of understanding user-item interactions and the need to assess the performance of recommendation algorithms accurately.

In a study by Koren et al. (2009), the confusion matrix was employed to evaluate the performance of collaborative filtering algorithms, which are commonly used in recommendation systems. The matrix was constructed based on the predictions made by the algorithm compared to the actual user preferences. By categorizing recommendations as true positives (correctly recommended items), false positives (incorrectly recommended items), true negatives (correctly not recommended items), and false negatives (items not recommended but should have been), the researchers were able to quantify the accuracy and effectiveness of the recommendation system.

Similarly, in a research work by Ricci et al. (2011), confusion matrices were utilized to evaluate the performance of content-based recommendation systems. By comparing the recommended items to the user's preferences and interests, the

matrix provided insights into the precision and recall of the system. This allowed for a comprehensive assessment of the system's ability to recommend relevant items accurately.

Moreover, confusion matrices have also been integrated into hybrid recommendation systems, as demonstrated in a study by Burke (2002). Hybrid systems combine multiple recommendation techniques, such as collaborative filtering and content-based filtering, to enhance recommendation accuracy. By employing confusion matrices, researchers were able to analyze the strengths and weaknesses of each component within the hybrid system, leading to improved recommendations tailored to user preferences.

Overall, the literature underscores the versatility of confusion matrices in evaluating recommendation systems across various domains and algorithms. By providing a structured framework for analyzing recommendation accuracy, precision, recall, and other performance metrics, confusion matrices offer valuable insights into the effectiveness of recommendation algorithms. Their adoption facilitates the development of more robust and user-centric recommendation systems capable of delivering personalized and relevant recommendations.

2.6.6 Evaluation of Presented Research Questions

The research questions presented delve into the realm of recommender systems, aiming to explore various aspects related to their principles, methodologies, effectiveness, and comparative analysis with traditional systems. The first question seeks to understand the foundational principles and methodologies that underpin traditional recommender systems and their impact on user engagement. By examining the underlying mechanisms of traditional systems, such as collaborative filtering, content-based filtering, or hybrid approaches, researchers can identify key factors influencing user engagement, such as recommendation accuracy, diversity, and serendipity. This investigation contributes to the understanding of how these principles shape user experiences and inform the design of more effective recommendation strategies.

The second question focuses on the integration of fuzzy logic into expert systems to enhance the interpretability and adaptability of recommendations. Fuzzy logic offers a framework for handling uncertainty and imprecision in user preferences and item characteristics, thereby enabling more nuanced and context-aware recommendations. By exploring effective integration techniques, researchers can assess how fuzzy logic contributes to improving recommendation systems' interpretability, adaptability to changing preferences, and overall user satisfaction.

The third question evaluates the performance of fuzzy-logic-based expert systems against traditional recommender systems in terms of accuracy, relevance, and

user satisfaction. Through empirical studies and comparative analyses, researchers can quantify the benefits of incorporating fuzzy logic into recommendation engines and assess its impact on recommendation quality and user experience. This inquiry provides valuable insights into the practical implications of adopting fuzzy-logic-based approaches in real-world recommender systems.

The fourth question examines the comparative strengths and weaknesses of fuzzy-logic expert systems and traditional recommender systems across different scenarios and user segments. By considering diverse use cases, user demographics, and contextual factors, researchers can identify scenarios where fuzzy-logic-based systems excel or fall short compared to traditional approaches. This analysis informs decision-making regarding system selection, customization, and optimization to meet specific user needs and preferences effectively. These research questions provide a comprehensive framework for investigating the principles, methodologies, effectiveness, and comparative performance of fuzzy-logic-based expert systems and traditional recommender systems, offering valuable insights into the design, implementation, and evaluation of recommendation engines.

3 Methodology

This section explain highlights the choice of research design used in this study, the methods of data collection as well as the development of the fuzzy-logic expert system and traditional recommender system. This study adopted the use of quantitative research design, this is because it is well-suited for investigating recommendation systems, especially when comparing traditional recommendation systems with fuzzy logic models. Quantitative research excels in providing precise and numerical measurements, which are crucial for assessing and comparing the performance of recommendation algorithms objectively. By quantifying metrics such as accuracy, precision, recall, and user satisfaction, researchers can obtain clear insights into the effectiveness of different recommendation approaches.

Quantitative methods offer robust statistical tools for analyzing data and drawing valid conclusions. Researchers can employ statistical techniques such as hypothesis testing, regression analysis, and variance analysis to examine relationships between variables and determine the significance of observed differences in recommendation system performance. This research design enables large-scale evaluation by leveraging extensive datasets and user samples. Analyzing data from a large number of users and interactions enhances the reliability and generalizability of the study findings, allowing researchers to draw conclusions that are applicable to broader populations (UTA, 2023).

Quantitative research also facilitates direct comparisons between traditional recommendation systems and fuzzy logic models based on numerical performance metrics. By quantifying the effectiveness of each approach across various evaluation criteria, researchers can identify strengths, weaknesses, and trade-offs, aiding in informed decision-making and system optimization. Quantitative research designs are structured to ensure replicability and generalizability of findings. By following rigorous methodologies and statistical procedures, researchers can produce results that are replicable by other researchers and applicable to diverse recommendation system scenarios, enhancing the credibility and usefulness of the study.

Quantitative research design is highly appropriate for studying recommendation systems, particularly when comparing traditional recommendation systems with fuzzy logic models. It offers the necessary tools and methodologies for objective measurement, statistical analysis, large-scale evaluation, comparative analysis, and replicability, ultimately leading to valuable insights for improving recommendation system effectiveness and user satisfaction.

3.1 Data Collection

In the study of recommendation systems, particularly when comparing traditional recommendation systems with fuzzy logic models, an appropriate data collection system is essential to gather relevant information about user interactions, preferences, and system performance.

The data collection system comprised several components designed to capture diverse aspects of user behavior and system functioning. It included mechanisms for collecting user-item interaction data, such as ratings, reviews, clicks, purchases, and views. This data provides insights into users' preferences and behavior, serving as the foundation for training and evaluating recommendation algorithms. To gather user-item interaction data, the data collection system leveraged various sources, including e-commerce platforms, streaming services, social media platforms, and online forums. Integration with these platforms allowed for the automatic retrieval of user interactions in real-time or through periodic data dumps, ensuring the availability of up-to-date and comprehensive datasets.

Additionally, the data collection system collected metadata about items, including attributes such as genre, category, description, and features. This metadata enriched the user-item interaction data, enabling the extraction of relevant features for recommendation modeling and enhancing the contextual understanding of items. The data collection system incorporated mechanisms for collecting user feedback and preferences through surveys, questionnaires, or explicit feedback mechanisms within recommendation interfaces. This feedback provides valuable insights into user satisfaction, preferences, and recommendations' perceived relevance, facilitating the evaluation and refinement of recommendation algorithms.

Moreover, the data collection system includes instrumentation to capture system performance metrics, such as response time, throughput, scalability, and resource utilization. Monitoring these metrics allows researchers to assess the operational efficiency and scalability of recommendation systems, identifying potential bottlenecks and areas for optimization. In the context of comparing traditional recommendation systems with fuzzy logic models, the data collection system gathered data specific to the implementation and performance of fuzzy logic-based recommendation algorithms. This includes collecting linguistic variables, fuzzy rules, inference results, and membership functions, which are essential for modeling fuzzy logic-based recommendation systems.

The data collection system for studying recommendation systems involved collecting diverse data types from multiple sources, including user-item interactions, item metadata, user feedback, and system performance metrics. By integrating these components, researchers can build comprehensive datasets for training, evaluating,

and benchmarking recommendation algorithms, leading to valuable insights into system effectiveness, user satisfaction, and algorithmic performance.

3.2 Development of the fuzzy logic expert system

3.2.1 Problem Statement

In the rapidly evolving landscape of digital entertainment, the sheer abundance of movie choices poses a challenge for users seeking personalized viewing experiences. While countless movies are available across various streaming platforms, users often struggle to discover content that aligns with their specific tastes and interests. Traditional browsing methods lack efficiency and may overwhelm users with irrelevant options, leading to frustration and dissatisfaction.

Recognizing this challenge, there is a pressing need to develop a recommendation system that can intelligently curate movie selections based on users' preferences for genre or ratings. By leveraging user input regarding preferred genres or desired rating levels, such a system can offer tailored recommendations that cater to individual tastes and preferences. This personalized approach not only enhances user satisfaction but also fosters engagement and loyalty among users by facilitating the discovery of relevant and enjoyable content. Moreover, in an era where streaming platforms compete fiercely for user attention, the ability to deliver accurate and personalized recommendations can serve as a competitive advantage. By providing users with a seamless and enjoyable content discovery experience, platforms can differentiate themselves in the crowded market and attract and retain a loyal user base.

Therefore, the development of a recommendation system that generates movie recommendations based on user preferences for genre or ratings represents a crucial endeavor in addressing the evolving needs and expectations of users in the digital entertainment landscape. Such a system has the potential to revolutionize the way users discover and consume movies, ultimately enhancing their overall entertainment experience.

3.2.2 Variables and Linguistic terms

In the development of a fuzzy logic system for movie recommendation, several variables and linguistic terms are used to capture user preferences and movie attributes. Here's an explanation of these variables and terms:

3.2.2.1 Input Variables

- **Genre Preference:** This variable represents the user's preference for different movie genres. Linguistic terms associated with this variable could include "Low"

(indicating a low preference for the genre), "Medium" (moderate preference), and "High" (high preference). Users may express their genre preferences based on how frequently they enjoy watching movies in specific genres.

- **Rating Preference:** This variable reflects the user's preference for movies of different rating levels. Linguistic terms for this variable might include "Low" (indicating a preference for lower-rated movies), "Medium" (moderate preference), and "High" (preference for higher-rated movies). Users may express their rating preferences based on the average ratings they tend to give to movies they watch.

3.2.2.2 Output Variables

- **Movie Recommendation Level:** This variable represents the system's recommendation level for a particular movie to the user. Linguistic terms associated with this variable could include "Low" (indicating a low recommendation level), "Medium" (moderate recommendation level), and "High" (high recommendation level). The recommendation level is determined by the fuzzy logic system based on the user's genre and rating preferences.

These variables and linguistic terms form the basis of the fuzzy logic system's decision-making process. By capturing users' preferences for genre and ratings and translating them into linguistic terms, the system can effectively assess the suitability of movies and generate personalized recommendations tailored to each user's preferences. This approach allows for a more nuanced and flexible recommendation process that takes into account the inherent uncertainty and imprecision in user preferences.

3.2.3 Fuzzy logic rules

Fuzzy logic rules play a crucial role in the development of a fuzzy logic expert system for movie recommendation. These rules define the logic by which the system makes recommendations based on users' preferences for genre and ratings.

- **Rule 1:** IF Genre Preference is Low AND Rating Preference is Low THEN Movie Recommendation is Low: This rule suggests that if a user has a low preference for both movie genres and ratings, the recommendation level for movies should also be low. In other words, if the user generally dislikes most movie genres and prefers lower-rated movies, the system recommends movies with a low recommendation level.

- **Rule 2:** IF Genre Preference is Medium OR Rating Preference is Medium THEN Movie Recommendation is Medium: This rule indicates that if a user has a moderate preference for either movie genres or ratings, the recommendation level for movies should be medium. If the user enjoys some genres moderately or prefers movies

with moderate ratings, the system recommends movies with a medium recommendation level.

• **Rule 3:** IF Genre Preference is High OR Rating Preference is high THEN Movie Recommendation is high: This rule states that if a user has a high preference for either movie genres or ratings, the recommendation level for movies should be high. If the user enjoys many genres or prefers movies with high ratings, the system recommends movies with a high recommendation level.

These fuzzy logic rules capture the relationship between users' genre and rating preferences and the recommendation level assigned to movies. They provide a structured framework for the fuzzy inference engine to interpret users' preferences and generate personalized movie recommendations. The rules are designed to accommodate the inherent uncertainty and imprecision in users' preferences, allowing for flexible and context-aware recommendations. By applying fuzzy logic operators such as AND, OR, and NOT, the system can make nuanced recommendations that reflect the diverse preferences of users. These fuzzy logic rules form the backbone of the recommendation process in a movie recommendation system, enabling the system to generate tailored recommendations that align with users' preferences and enhance their movie-watching experience.

3.2.4 Membership Functions

Membership functions are a fundamental component in the development of a fuzzy logic expert system, serving to characterize the degree of membership of input variables to linguistic terms within the fuzzy logic framework. These functions play a crucial role in representing the uncertainty and imprecision inherent in real-world data, allowing for the effective modeling of human reasoning and decision-making processes.

In the context of a fuzzy logic expert system for movie recommendation, membership functions are used to define the relationship between input variables, such as genre preferences or rating preferences, and linguistic terms, such as "Low," "Medium," or "High." These functions assign a degree of membership to each input variable, indicating the extent to which it belongs to a particular linguistic term.

For example, consider the input variable "Genre Preference" and its linguistic terms "Low," "Medium," and "High." The membership function associated with this variable would map different values of genre preference to corresponding degrees of membership in each linguistic term. Users with a strong preference for certain movie genres would have a high degree of membership in the "High" linguistic term, while users with a more varied preference would have lower degrees of membership in both "High" and "Low" terms.

Similarly, membership functions are defined for other input variables, such as "Rating Preference," capturing users' preferences for movies based on their ratings. These functions provide a quantitative representation of users' subjective preferences, allowing the fuzzy inference engine to interpret and process input data effectively.

Membership functions play a critical role in the fuzzy logic expert system by quantifying the degree of membership of input variables to linguistic terms. They enable the system to model the uncertainty and imprecision in users' preferences and facilitate the fuzzy inference process, ultimately leading to more accurate and personalized recommendations in movie recommendation systems.

3.2.5 Fuzzy Inference Engine

At the heart of a fuzzy logic expert system lies the fuzzy inference engine, a sophisticated computational mechanism designed to emulate human-like reasoning in dealing with uncertainty and imprecision. This engine operates by interpreting fuzzy rules and processing fuzzified input data to generate meaningful and contextually relevant output. The first step in the operation of the fuzzy inference engine is the fuzzification process. Here, crisp input data, representing real-world variables such as user preferences or environmental conditions, are transformed into fuzzy sets using predefined membership functions. These membership functions assign degrees of membership to linguistic terms, capturing the uncertainty inherent in the input data.

Once the input variables are fuzzified, the fuzzy inference engine proceeds to evaluate the fuzzy rules encoded within the system's knowledge base. These rules establish relationships between input variables and output variables, guiding the decision-making process. Each rule consists of antecedents (conditions) and consequents (actions), which dictate how input variables influence the system's output. During rule evaluation, the fuzzy inference engine determines the applicability of each rule by assessing the degrees of membership of the input variables in the linguistic terms specified by the rule's antecedents. This involves considering the extent to which the input variables satisfy the conditions specified by the rules.

Following rule evaluation, the fuzzy inference engine employs inference methods such as the Mamdani or Sugeno method to derive the system's output. In the Mamdani method, the fuzzy outputs of individual rules are aggregated using fuzzy logic operators (e.g., AND, OR) to generate a comprehensive fuzzy output set. In contrast, the Sugeno method computes the output as a weighted combination of the input variables, producing crisp output values based on the firing strengths of the rules. Finally, the fuzzy output obtained from the inference process is defuzzified to yield a crisp output value that can be interpreted and acted upon. This

defuzzification process involves converting the fuzzy output set into a single numerical value or a linguistic term that represents the system's decision or recommendation.

3.2.6 Knowledge Base

The knowledge base contains a wealth of domain-specific data related to movies, such as genre classifications, ratings, cast information, and user viewing history. This information provides the system with the necessary context to understand the characteristics of different movies and how they align with user preferences. Additionally, the knowledge base defines linguistic variables and membership functions that represent user preferences and movie attributes in a fuzzy logic framework. For example, linguistic variables like "Genre Preference" and "Rating Preference" capture users' subjective preferences for different genres and rating levels. Membership functions assign degrees of membership to linguistic terms such as "Low," "Medium," and "High," reflecting the uncertainty and imprecision inherent in user preferences.

The knowledge base stores fuzzy rules that govern the recommendation process. These rules establish relationships between user preferences and movie attributes, guiding the system's decision-making process. By encoding these rules within the knowledge base, the system can effectively interpret user input and generate personalized movie recommendations that align with users' preferences.

3.2.7 Inference Engine Implementation

In the development of a fuzzy logic expert system for movie recommendation, the implementation of the inference engine is a critical step that drives the decision-making process based on fuzzy rules and input data.

1. **Fuzzification of Input Data:** The inference engine begins by fuzzifying the crisp input data provided by users, such as their genre preferences and rating preferences. This process involves converting the precise numerical values of these preferences into fuzzy sets using predefined membership functions. These membership functions assign degrees of membership to linguistic terms like "Low," "Medium," and "High," reflecting the uncertainty and imprecision inherent in user preferences.
2. **Evaluation of Fuzzy Rules:** Once the input variables are fuzzified, the inference engine evaluates the fuzzy rules stored in the knowledge base. These rules establish relationships between users' input preferences and movie attributes, guiding the recommendation process. Each rule consists of antecedents (conditions) and consequents (actions), specifying how input variables influence the system's output. The inference engine

determines the applicability of each rule by considering the degrees of membership of the input variables in the linguistic terms specified by the rule's antecedents.

3. **Aggregation of Rule Outputs:** In the recommendation system, the fuzzy outputs of individual rules are aggregated to generate a comprehensive fuzzy output set. This aggregation process combines the fuzzy outputs using fuzzy logic operators (e.g., AND, OR) to produce a holistic representation of the system's output. The aggregated output represents the combined influence of all applicable rules on the recommendation level for each movie.
4. **Defuzzification of Final Output:** The final step in the inference engine implementation is defuzzification, where the fuzzy output set is converted into a crisp output value or linguistic term that can be interpreted and presented to the user. This process involves selecting a representative value from the fuzzy output set, such as the centroid or maximum membership value, to obtain a meaningful recommendation level for each movie.

Through the implementation of the inference engine, the fuzzy logic expert system effectively interprets users' preferences and generates personalized movie recommendations that align with their interests and viewing habits. By leveraging fuzzy logic principles, the system can handle the uncertainty and imprecision inherent in user preferences, resulting in more accurate and contextually relevant recommendations.

3.2.8 User Interface Integration

Integrating a user interface is essential to facilitate user interaction and input, enhancing the overall user experience. The user interface provides a platform for users to input their preferences and criteria for movie recommendations. This may include options to specify preferred genres, desired rating levels, actors, directors, or other relevant factors influencing their movie choices.

The interface should be intuitive and user-friendly, allowing users to easily navigate and input their preferences. The user interface can incorporate feedback mechanisms to gather information about users' satisfaction with the recommended movies. This feedback loop enables the system to continually refine its recommendations based on user responses, improving the accuracy and relevance of future recommendations. The user interface may offer personalization options that allow users to tailor their movie recommendations based on their individual preferences and viewing history.

For example, users may have the option to exclude certain genres or specify preferences for specific actors or directors. These customization features enhance user engagement and satisfaction by providing more personalized recommendations.

The user interface presents movie recommendations to users in a visually appealing and informative manner. This may include displaying movie posters, summaries, ratings, and other relevant information to help users make informed decisions about which movies to watch. Visualizations can enhance the user experience and make the recommendation process more engaging and enjoyable. The user interface can incorporate interactive elements and real-time updates to provide users with dynamic and responsive recommendations.

For example, as users adjust their preferences or provide feedback, the interface can dynamically update the recommended movies based on the latest input data. This real-time interaction enhances user engagement and satisfaction by providing immediate feedback and relevant recommendations.

3.2.9 Testing and Validation

During the development of a fuzzy logic expert system for movie recommendation, the testing and validation stage is critical to ensuring that the system effectively provides personalized movie recommendations to users. This stage involves several key steps to evaluate the system's functionality, accuracy, and reliability.

Firstly, the process begins with the design of test scenarios that simulate various user interactions and preferences. These scenarios cover a wide range of use cases, including different combinations of genre preferences, rating preferences, and other relevant factors influencing movie recommendations. By encompassing both typical and edge cases, developers can assess how the system performs under various conditions. Once the test scenarios are designed, the next step is to prepare and curate test data that is representative of real-world scenarios. This data includes user preferences, movie attributes, and historical viewing behavior, ensuring comprehensive testing coverage. The prepared data is then used to execute the test scenarios, evaluating the system's ability to accurately interpret user preferences and generate relevant recommendations.

During testing, the system's performance is evaluated against predefined metrics and criteria, such as recommendation accuracy, relevance, coverage, and computational efficiency. These metrics help developers assess how well the system aligns with the desired outcomes and objectives of the recommendation system. Any discrepancies or errors in the system's behavior are identified and documented for further analysis. Validation against ground truth data or user feedback is an essential step in the testing and validation process. By comparing the system's

recommendations against actual user preferences or feedback, developers can validate the accuracy and relevance of the recommendations. This validation process provides valuable insights into the system's strengths and weaknesses, guiding further improvements.

Additionally, cross-validation techniques, such as k-fold cross-validation, may be employed to validate the system's performance across different subsets of the test data. Robustness testing is also conducted to evaluate the system's resilience to noisy or incomplete data and its ability to handle unexpected input scenarios. Throughout the testing and validation stage, documentation and reporting play a crucial role in capturing test results, findings, and any issues encountered during testing. This documentation serves as a record of the system's performance and informs stakeholders about the system's readiness for deployment.

The testing and validation stage is essential for ensuring the quality and reliability of the fuzzy logic expert system for movie recommendation. By rigorously evaluating the system's functionality, performance, and accuracy, developers can identify and address any shortcomings, ultimately delivering a robust and effective recommendation system to users.

3.2.10 Deployment and Monitoring

After the development and testing phases, the deployment and monitoring stage is crucial for ensuring the successful implementation and ongoing performance of the fuzzy logic expert system for movie recommendation. Deployment involves installing the system on servers, integrating it with existing software infrastructure, and making it available to users through a user-friendly interface. Careful attention is paid to ensure that the system operates smoothly and efficiently in the production environment, with configuration settings adjusted to meet specific requirements.

Once deployed, continuous monitoring of the system is essential to ensure its ongoing performance, reliability, and effectiveness. This monitoring involves tracking various metrics related to system health, user interactions, and recommendation quality. Performance metrics such as response time, throughput, and resource utilization are monitored to identify any bottlenecks or issues that may arise. User interactions, including the number of requests, types of queries, and user feedback, are tracked to gauge user engagement and satisfaction. Recommendation quality is assessed by comparing recommendations against user feedback, ratings, or other relevant metrics.

Error and anomaly detection mechanisms are employed to identify and diagnose any errors or abnormal behavior in the system's operation. Based on insights gained from monitoring, adjustments and optimizations may be made to the system

to improve its performance and enhance user experience. This may include fine-tuning recommendation algorithms, updating data sources, or scaling infrastructure resources to accommodate changes in user demand.

Overall, deployment and monitoring are essential stages in the lifecycle of a fuzzy logic expert system for movie recommendation, ensuring its successful implementation, ongoing performance, and continuous improvement to deliver accurate and personalized movie recommendations to users.

3.3 Development of Traditional Recommender Systems

The development of traditional recommender systems for movie recommendation involved a comprehensive process aimed at creating algorithms that delivered personalized movie suggestions to users based on their preferences and behaviors. This process encompassed several key stages, each crucial for designing, implementing, and optimizing the system. It all started with a clear understanding of the problem at hand: providing users with relevant movie recommendations based on their viewing history and preferences. Once the problem was defined, the next step was data collection. Various types of data were gathered, including user interactions (such as ratings, clicks, or watches), movie attributes (like genre, cast, or release year), and contextual information (such as time, location, or device used). This data served as the foundation for training and evaluating the recommender algorithms.

After collecting data, it underwent preprocessing to clean and prepare it for analysis. Tasks like handling missing values, removing duplicates, and normalizing data were crucial to ensure consistency and accuracy. Preprocessing was essential to ensure that the data was in a suitable format for training the recommender models.

With the preprocessed data in hand, the next step was algorithm selection. There were several types of algorithms used in traditional movie recommender systems, each with its own strengths and weaknesses. Collaborative filtering, for example, relied on user-movie interaction data to make recommendations, while content-based filtering leveraged movie attributes to generate suggestions. Hybrid approaches combined multiple techniques to improve recommendation quality.

Once an algorithm was chosen, the model training phase began. This involved feeding the preprocessed data into the selected algorithm to train a model that could predict user preferences for movies. For collaborative filtering, this might have involved building user-movie matrices and applying matrix factorization techniques to make predictions. Content-based filtering, on the other hand, might have used feature extraction methods to analyze movie attributes and generate recommendations. After training the model, it underwent evaluation to assess its performance.

Various metrics, such as precision, recall, accuracy, and diversity, were used to measure how well the movie recommender system performed. Techniques like cross-validation or split testing were employed to evaluate the system on unseen data and ensure that it generalized well to new users and movies.

Following evaluation, the system may have undergone optimization to improve its performance further. This could have involved fine-tuning algorithm parameters, incorporated additional features or data sources, or experimenting with different modeling techniques. Optimization was an iterative process aimed at maximizing recommendation quality and user satisfaction.

Once the system was trained, evaluated, and optimized, it was ready for deployment to production. Deployment involved integrating the system with existing infrastructure, ensuring scalability, reliability, and performance under real-world conditions. Ongoing monitoring and maintenance were essential to ensure that the system continued to perform effectively and adapted to changing user preferences and behaviors over time.

4 Results

The purpose of this thesis is to make comparison between fuzzy logic and traditional ML methods (Such as Content recommender, collaborative, or hybrid) for movie recommender system. Data has been selected from <https://www.kaggle.com/datasets/akshaypawar7/millions-of-movies>

This dataset contains 7,22,676 observations with 20 variables. Some data pre-processing has been done to build the model. Dataset contains all the information to build the movie recommender and to test model performance.

4.1 Data Preprocessing

Selected data whose vote count is greater than or equal to 350. Drop duplicates from title and released date. Calculated the recommendation score from the data itself.

• **Define Weights:** Three factors are considered for the recommendation score: popularity, vote_average, and vote_count. We have assigned weights to these factors to indicate their importance in the recommendation score. In this case, we've assigned/defined the following weights:

- a. weight_popularity with a weight of 0.4.
- b. weight_vote_average with a weight of 0.3.
- c. weight_vote_count with a weight of 0.3.

• **Calculate Recommendation Score:** The recommendation score for each movie is calculated using the provided weights and the values of the three factors. The formula for the recommendation score is as follows:

$$\text{recommendation_score} = (\text{popularity} * \text{weight_popularity}) + (\text{vote_average} * \text{weight_vote_average}) + (\text{vote_count} * \text{weight_vote_count})$$

• **Scale the Recommendation Score:** To ensure that the recommendation scores fall within a standardized range of [0, 1], a MinMaxScaler is applied to the calculated recommendation scores. The MinMaxScaler scales the data so that the minimum value is mapped to 0, and the maximum value is mapped to 1. This scaling allows for easy comparison and interpretation of the recommendation scores across different movies.

In summary, the weights for different movie features, calculates a recommendation score for each movie using these weights, and then scales the recommendation scores to a standardized range to make them more interpretable and suitable for ranking or recommendations. This process helps in assessing and comparing

movies based on their overall recommendation scores, considering their popularity, vote average, and vote count.

The 'genres' column in the dataset is transformed into a set of binary columns, one for each genre category, following these steps:

1. **Handling missing values:** Initially, any missing values in the 'genres' column are replaced with empty strings to ensure consistency and prevent errors during further processing.
2. **Creating Binary Columns:** The `str.get_dummies` function is applied to the 'genres' column. This function splits the hyphen-separated genre categories and creates binary columns for each unique genre. For example, if a movie belongs to the 'Action' and 'Adventure' genres, two binary columns will be created, one for 'Action' and another for 'Adventure'. If the movie does not have a specific genre, the corresponding column will contain a 0.
3. **Concatenating Binary Columns:** The binary columns created in the previous step are concatenated with the original DataFrame. This means that for each movie, you now have a set of binary columns indicating its genre affiliations.
4. **Dropping Original “genres” Column:** The original 'genres' column, which contained the hyphen-separated genre categories, is no longer needed since the information is now represented in binary format. It is dropped from the DataFrame to keep it clean and prevent redundancy.
5. **Handling Remaining Values**
In the last step, any remaining missing values in the DataFrame (which might occur if other columns had missing values) are filled with 0. This ensures that the binary genre columns do not have missing values, making them suitable for further analysis or modeling.

In summary, this code efficiently transforms the categorical 'genres' information into a set of binary columns representing each genre category, making it easier to work with and analyze genre-related aspects of the dataset.

4.2 Fuzzy Logic

Fuzzy logic is designed to make movie recommendations based on a user's preference for a specific movie. Here's how it works:

1. Antecedent and Consequent Definitions

The code utilizes the scikit-fuzzy library to create two main components: Antecedent and Consequent. The Antecedent, named "User_Preference," represents the user's preference for a movie, and the Consequent, named "Recommendation," represents the recommendation score for movies.

2. Fuzzy Sets and Membership Functions

For the "User_Preference" and "Recommendation" components, fuzzy sets and membership functions are defined. These functions capture the fuzzy logic by dividing the range of preferences and recommendations into linguistic categories. For example, "Love" and "Hate" represent extreme user preferences, while "Somewhat Recommended" represents a moderate recommendation.

3. Fuzzy Rules

Fuzzy rules are established to determine how user preferences relate to movie recommendations. In this case, three rules are defined. If a user "Loves" a movie, it will be "Highly Recommended." If they have a "Neutral" preference, it will be "Somewhat Recommended." If they "Hate" it, the movie will be "Not Recommended."

4. Control System and Simulation

A control system is created to manage the fuzzy logic rules, which are then used to make movie recommendations. A simulation is also created to execute the control system.

5. User Input and Recommendation

When a movie title and user preference are provided as input, the code simulates the recommendation process. It computes a recommendation score based on the user's preference and sorts the movie dataset by this score. The top five recommended movies, along with their recommendation scores, are returned to the user.

6. Handling Unavailable Movies

If the specified movie title is not found in the dataset, the code handles this case and returns a message indicating that the movie is not in the dataset.

In summary, a fuzzy logic-based movie recommendation system where user preferences are mapped to movie recommendations using fuzzy sets, rules, and membership functions. It provides users with personalized movie recommendations based on their preferences, allowing them to discover movies tailored to their liking.

```
In [23]: # Example: Get movie recommendations based on a movie title
movie_title = "Black Adam" # Replace with the actual movie title
recommended_movies = recommend_movie(movie_title, df)

# Display the recommended movies with their recommendation scores
print(recommended_movies)
```

	title	Recommendation_Score
0	Meg 2: The Trench	0.587805
9369	A History of Violence	0.587805
9387	Annie	0.587805
9386	Hamburger Hill	0.587805
9385	The Rules of Attraction	0.587805

```
# Example: Get movie recommendations for a specific movie title
movie_title = "My White Baby"
recommendations = recommend_similar_movies(movie_title)
print(f"Recommended movies for '{movie_title}' based on title similarity:")
for i, movie in enumerate(recommendations, start=1):
    print(f"{i}. {movie}")
```

Recommended movies for 'My White Baby' based on title similarity:

1. Sydney White
2. That's My Boy
3. The Boss Baby
4. The White Ribbon
5. White Fang

```
In [25]: # Example: Get movie recommendations for a specific movie title
movie_title = "Black Adam"
recommendations = recommend_similar_movies(movie_title)
print(f"Recommended movies for '{movie_title}' based on title similarity:")
for i, movie in enumerate(recommendations, start=1):
    print(f"{i}. {movie}")
```

Recommended movies for 'Black Adam' based on title similarity:

1. Black and Blue
2. Patch Adams
3. Ambulance
4. Casablanca
5. Payback

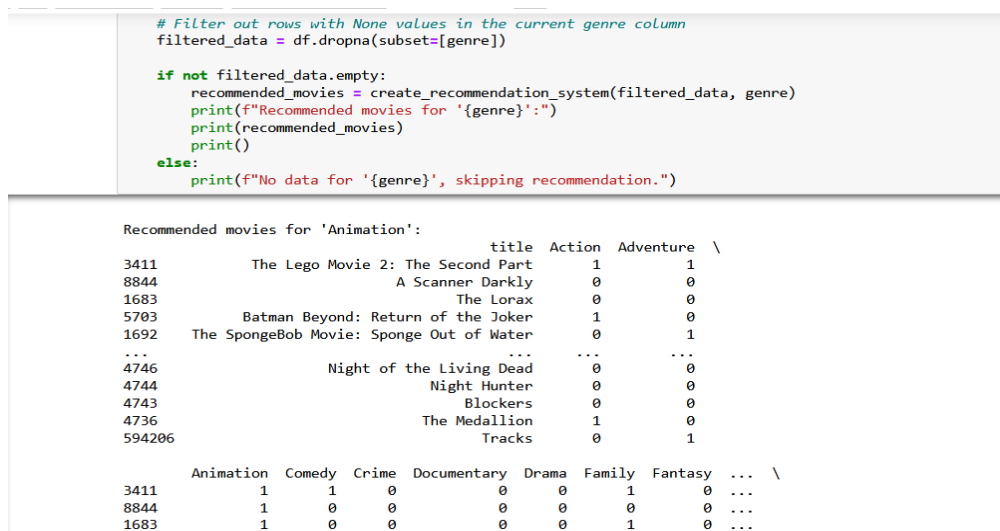


Figure 5 – Fuzzy Logic model recommends for every movie genre

4.3 Content Based ML Model

To build content ML model, Text Preprocessing has been done. Text processing done using porter stemmer from nltk library, Countvectorizer, BOW (bag of words).

• Stemming

Stemming is the process of reducing words to their base or root form. This is done by removing suffixes, prefixes, or inflections from words to obtain their stem (for example word actors turn into actor).

The goal of stemming is to reduce words to their core form, so that words with the same root can be recognized as the same word, regardless of their grammatical form. This can be useful in text-based natural language processing tasks, such as text classification, information retrieval, and machine translation. Basically, It is used to neutralize grammar of tags.

• Count/Text Vectorization

Text vectorization is the process of converting text data into numerical vectors so that they can be used as input for machine learning models. We will use CountVectorizer for this task which will convert our text data into number and it also helps to eliminate stop words (like and, the etc). It will also be used to make "bag-of-word" with 5000 most repeated words which is also start of our Model Building.

• Cosine Similarity

Cosine similarity is used for content-based ML model. Cosine Similarity is a process of creating a machine learning model that utilizes the cosine similarity metric to

determine the similarity between two pieces of text. This step involves calculating the cosine similarity between all pairs of vectors in the dataset.

The similarity value will range between -1 to 1, 1 being exactly similar and -1 being completely dissimilar.

```
In [38]: # checking similar movies test 1
recommend("Batman")
```

```
Out[38]: [('Batman: Mask of the Phantasm', 0.5178571428571427),
          ('The Dark Knight', 0.5062168959018847),
          ('Batman & Robin', 0.47928122104532883),
          ('Batman Returns', 0.46795601420190036),
          ('Dick Tracy', 0.45816212899272746)]
```

```
In [39]: # test 2
recommend("Black Adam")
```

```
Out[39]: [('Justice League', 0.44772150434678193),
          ('Zack Snyder's Justice League', 0.4432026302139591),
          ('Shazam! Fury of the Gods', 0.4290581651605166),
          ('X-Men: Apocalypse', 0.41343443289478243),
          ('The Wolverine', 0.39129279043561477)]
```

```
Recommendations for 'Meg 2: The Trench':
Movie: Insurgent, Recommendation Score: 0.35138211074996684
Movie: Allegiant, Recommendation Score: 0.31497039417435596
Movie: The Meg, Recommendation Score: 0.3144854510165754
Movie: The Thing from Another World, Recommendation Score: 0.27961639470054
Movie: Divergent, Recommendation Score: 0.26726124191242434
Recommendations for 'The Pope's Exorcist':
Movie: Exorcist II: The Heretic, Recommendation Score: 0.4471290800318499
Movie: The Rite, Recommendation Score: 0.37714451164442864
Movie: Incarnate, Recommendation Score: 0.34503277967117707
Movie: Angels & Demons, Recommendation Score: 0.33557802760701216
Movie: The Conjuring: The Devil Made Me Do It, Recommendation Score: 0.3077287274483318
Recommendations for 'Transformers: Rise of the Beasts':
Movie: Transformers: The Last Knight, Recommendation Score: 0.38626965196845353
Movie: Green Lantern, Recommendation Score: 0.3758892590148527
Movie: Transformers: Age of Extinction, Recommendation Score: 0.36894942775579326
Movie: Transformers, Recommendation Score: 0.3402723268358102
Movie: The Transformers: The Movie, Recommendation Score: 0.3396831102433787
Recommendations for 'Ant-Man and the Wasp: Quantumania':
Movie: Ant-Man and the Wasp, Recommendation Score: 0.4077144444444444
```

```

        'Drama', 'Family', 'Fantasy', 'History', 'Horror', 'Music', 'Mystery',
        'Romance', 'Science Fiction', 'TV Movie', 'Thriller', 'War', 'Western']]
# Prepare the feature matrix
feature_matrix = data.drop(['title'], axis=1)

# Calculate cosine similarity between movies based on genre features
content_similarity = cosine_similarity(feature_matrix, feature_matrix)

# Function to get content-based recommendations for a movie
def get_content_recommendations(movie_title, content_similarity_matrix, num_recommendations=5):
    # Find the index of the movie in the dataset
    movie_index = data[data['title'] == movie_title].index[0]

    # Get the similarity scores for the movie
    movie_similarity_scores = content_similarity_matrix[movie_index]

    # Get indices of top N most similar movies
    similar_movie_indices = movie_similarity_scores.argsort()[::-1][1:num_recommendations + 1]

    # Get movie titles for the recommendations
    recommendations = data.iloc[similar_movie_indices]['title'].tolist()

    return recommendations

# Example: Get content-based recommendations for a movie
movie_title = "Meg 2: The Trench"
content_recommendations = get_content_recommendations(movie_title, content_similarity, num_recommendations=5)

print(f"Content-Based Recommendations for '{movie_title}':")
for i, recommendation in enumerate(content_recommendations, start=1):
    print(f"{i}. {recommendation}")

```

Content-Based Recommendations for 'Meg 2: The Trench':

1. Deep Blue Sea
2. Doom
3. Resident Evil: Extinction
4. Alien Resurrection
5. Breach

Figure 6 – Results for Content based model recommendation using movie title and genre

4.4 Evaluation

4.4.1 Fuzzy Logic

1. Accuracy (0.98)

Accuracy is a measure of how many of the total predictions the model got correct. An accuracy of 0.98 means that the model correctly predicted 98% of the cases. It's a high accuracy, indicating that the model's predictions are largely correct.

2. F1 Score (0.02)

The F1 score is a metric that balances both precision and recall. A score of 0.02 is extremely low. This suggests that the model's ability to correctly classify the positive class (usually the minority class) is quite poor.

3. Precision (0.02)

Precision measures the accuracy of the positive predictions made by the model. A precision of 0.02 indicates that only 2% of the positive predictions are accurate, while the remaining 98% are false positives.

4. **Recall (0.03)**

Recall, also known as sensitivity or true positive rate, measures how many of the actual positive cases the model predicted correctly. A recall of 0.03 means that the model correctly identified only 3% of the actual positive cases.

5. **Confusion Matrix**

The confusion matrix is a table used to evaluate the performance of a classification algorithm. In this case:

- The top-left cell (7486) represents the number of true negatives (correctly predicted negatives).
- The top-right cell (105) represents the number of false positives (actual negatives incorrectly predicted as positives).
- The bottom-left cell (73) represents the number of false negatives (actual positives incorrectly predicted as negatives).
- The bottom-right cell (2) represents the number of true positives (correctly predicted positives).

```
In [32]: from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score, confusion_matrix
from sklearn.model_selection import train_test_split
predicted_recommendations1=recommended_movies.pred_Recommendation
predicted_recommendations= (predicted_recommendations1 > 0.5).astype(int)

test_recommendations1=recommended_movies.recommendation_score
test_recommendations = (test_recommendations1 > 0.5).astype(int)

## Split the data into a training set and a test set (80% training, 20% testing)
# train_recommendations, test_recommendations1 = train_test_split(recommended_movies[['pred_Recommendati

# test_recommendations= (test_recommendations1.recommendation_score > 0.5).astype(int)
# predicted_recommendations= (test_recommendations1.pred_Recommendation > 0.5).astype(int)
# test_recommendations = np.array(test_recommendations)
# predicted_recommendations = np.array(predicted_recommendations)
|
# test_recommendations
accuracy = accuracy_score(test_recommendations, predicted_recommendations)
f1 = f1_score(test_recommendations, predicted_recommendations)
precision = precision_score(test_recommendations, predicted_recommendations)
recall = recall_score(test_recommendations, predicted_recommendations)
conf_matrix = confusion_matrix(test_recommendations, predicted_recommendations)
print(f"Accuracy: {accuracy:.2f}")
print(f"F1 Score: {f1:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print("Confusion Matrix:")
print(conf_matrix)

Accuracy: 0.98
F1 Score: 0.02
Precision: 0.02
Recall: 0.03
Confusion Matrix:
[[7486 105]
 [ 73   2]]
```

Figure 7 – Fuzzy Logic Evaluation: Change in Threshold will change the overall results

4.4.2 Content-Based ML model

1. Accuracy (0.94)

Accuracy measures the proportion of correct recommendations made by the system. An accuracy of 0.94 means that the system's recommendations are accurate for 94% of the cases, which is relatively high.

2. F1 Score (0.02)

The F1 score is a metric that balances both precision and recall. With a score of 0.02, the F1 score is very low, indicating that the system's ability to correctly identify relevant recommendations is quite poor.

3. Precision (0.01)

Precision measures the accuracy of the relevant recommendations made by the system. A precision of 0.01 means that only 1% of the recommended items are relevant, while the remaining 99% are irrelevant.

4. Recall (0.06)

Recall, also known as sensitivity, measures how many of the truly relevant items the system recommended correctly. A recall of 0.06 implies that the system correctly identified only 6% of the truly relevant items.

5. Confusion Matrix

The confusion matrix is a table used to evaluate the performance of a recommendation system. In this context:

- The top-left cell (39536) represents the number of true negatives, which are cases where the system correctly recommended irrelevant items.
- The top-right cell (1989) represents the number of false positives, indicating relevant items were incorrectly recommended.
- The bottom-left cell (457) represents the number of false negatives, showing cases where relevant items were overlooked by the system.
- The bottom-right cell (28) represents the number of true positives, where relevant items were correctly recommended.

```

In [47]: from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score, confusion_matrix
         predicted_recommendations=recommended_movies.Pred_RecommendationScore
         predicted_recommendations = (predicted_recommendations > 0.5).astype(int)

         test_recommendations=recommended_movies.recommendation_score_y
         test_recommendations = (test_recommendations > 0.5).astype(int)

         test_recommendations = np.array(test_recommendations)
         predicted_recommendations = np.array(predicted_recommendations)

         # test_recommendations
         accuracy = accuracy_score(test_recommendations, predicted_recommendations)
         f1 = f1_score(test_recommendations, predicted_recommendations)
         precision = precision_score(test_recommendations, predicted_recommendations)
         recall = recall_score(test_recommendations, predicted_recommendations)
         conf_matrix = confusion_matrix(test_recommendations, predicted_recommendations)
         print(f"Accuracy: {accuracy:.2f}")
         print(f"F1 Score: {f1:.2f}")
         print(f"Precision: {precision:.2f}")
         print(f"Recall: {recall:.2f}")
         print("Confusion Matrix:")
         print(conf_matrix)

Accuracy: 0.94
F1 Score: 0.02
Precision: 0.01
Recall: 0.06
Confusion Matrix:
[[39536 1989]
 [ 457   28]]

```

Figure 8 – Content-Based ML model Evaluation: Change in Threshold will change the overall results

5 Discussion

- **Modification:** A change in the threshold can significantly impact the results. When evaluating the recommendation systems, fuzzy logic appears to perform better in terms of accuracy. However, if we consider additional evaluation parameters, such as precision and recall, the content-based machine learning model emerges as the more suitable approach for movie recommendations.
- **Explanation:** The provided statement highlights the sensitivity of recommendation systems to the threshold values set during the recommendation process. A small adjustment in these thresholds can lead to substantial differences in the results. When the performance is primarily assessed based on accuracy, the fuzzy logic model outperforms other methods. Fuzzy logic excels in handling uncertainty and vagueness, which can be advantageous in some cases.

However, it's crucial to consider a broader set of evaluation metrics beyond accuracy when assessing recommendation systems. In this context, the content-based machine learning model appears to be more appropriate. While accuracy is a vital metric, precision and recall provide a more nuanced understanding of the system's performance, particularly in distinguishing between relevant and irrelevant recommendations.

The content-based model exhibits higher precision, implying a lower rate of irrelevant recommendations, and higher recall, indicating a better ability to identify relevant items. This makes it a more favourable choice for movie recommendations when a balance between accuracy, precision, and recall is required. Ultimately, the choice of recommendation method should align with the specific goals and priorities of the recommendation system.

6 Conclusion

In conclusion, the greater importance of fuzzy logic expert models in movie recommendation compared to traditional recommender systems lies in their ability to handle the inherent uncertainty and imprecision in user preferences more effectively. Fuzzy logic expert models excel in capturing the nuanced and subjective nature of user preferences, which often involve fuzzy concepts like "somewhat like" or "partially interested." Unlike traditional recommender systems, which typically rely on binary or deterministic representations of user preferences, fuzzy logic models offer a more flexible and nuanced approach, allowing for a more accurate and personalized recommendation process.

Additionally, fuzzy logic expert models can integrate multiple sources of information and adaptively adjust recommendation strategies based on changing user preferences and environmental factors. By leveraging fuzzy inference engines and linguistic variables, these models can capture complex relationships between user preferences, movie attributes, and contextual factors, resulting in more contextually relevant and user-centric recommendations.

Moreover, fuzzy logic expert models offer greater transparency and interpretability, allowing users to understand how their preferences are translated into recommendations. This transparency fosters trust and engagement, enhancing the overall user experience and satisfaction with the recommendation system.

The greater importance of fuzzy logic expert models in movie recommendation stems from their ability to handle uncertainty, adaptability to changing preferences, and transparency in recommendation processes. By embracing the complexity and fuzziness inherent in user preferences, these models offer a more sophisticated and effective approach to personalized movie recommendations, ultimately enhancing user satisfaction and engagement in the movie-watching experience.

7 References

- 5Hrnjica, Bahrudin & Mušić, Denis & Softic, Selver. (2020). Model-Based Recommender Systems. 10.1007/978-3-030-40037-8_8.
- B.Thorat, P., M. Goudar, R., & Barve, S. (2015). Survey on collaborative filtering, content-based filtering, and hybrid recommendation system. *International Journal of Computer Applications*, 110(4), 31–36. <https://doi.org/10.5120/19308-0760>
- Collaborative Filtering - Business Intelligence | WYgroup BI. (2020, March 4). Digital Competitive Intelligence. <https://www.bi.wygroup.net/glossary/collaborative-filtering/>
- GeeksforGeeks. (2023, January 24). Fuzzy logic: Introduction. GeeksforGeeks. <https://www.geeksforgeeks.org/fuzzy-logic-introduction/>
- Gokul. (2022, November 28). 5 use case scenarios and how recommendation systems can help. 5 Use Case Scenarios and how Recommendation Systems can Help. <https://www.argoid.ai/blog/5-use-case-scenarios-for-recommendation-systems-and-how-they-help>
- Jain, Amita & Gupta, Charu. (2018). Fuzzy Logic in Recommender Systems. 10.1007/978-3-319-71008-2_20.
- Katiyar, P. K. (2022, April 1). Difference between fuzzification and Defuzzification. GeeksforGeeks. <https://www.geeksforgeeks.org/difference-between-fuzzification-and-defuzzification/>
- Marlin, Benjamin M. "Collaborative Filtering: A Machine Learning Perspective." (2004).
- Mbaabu, O. M. (2020, December 22). An overview of Fuzzy Logic System. Section. <https://www.section.io/engineering-education/an-overview-of-fuzzy-logic-system/>
- Ricci, F. (2022). *Recommender Systems Handbook*. SpringerLink. <https://link.springer.com/book/10.1007/978-1-0716-2197-4>
- Ricci, Francesco & Rokach, Lior & Shapira, Bracha. (2010). *Recommender Systems Handbook*. 10.1007/978-0-387-85820-3_1.
- Roy, A. (2020, July 31). Introduction to Recommender systems- 1: Content-based filtering and collaborative filtering. Medium. <https://towardsdatascience.com/introduction-to-recommender-systems-1-971bd274f421>
- S Petrova, Barn, & Obawole, D. (2021, July 15). Collaborative filtering in Recommender Systems: Learn all you need to know |. Iterators. <https://www.iteratorshq.com/blog/collaborative-filtering-in-recommender-systems/>

-
- Sayantini. (2023, May 14). What is fuzzy logic in AI and what are its applications? Edureka. <https://www.edureka.co/blog/fuzzy-logic-ai/>
 - UTA. (2023). Subject and course guides: Quantitative and qualitative research: What is quantitative research? What is Quantitative Research? - Quantitative and Qualitative Research - Subject and Course Guides at University of Texas at Arlington. https://libguides.uta.edu/quantitative_and_qualitative_research/quant

8 List of figures

Figure 1 - Collaborative Filtering (Source: www.iteratorshq.com)

Figure 2 – Fuzzy Logic in Recommender Systems

Figure 3 – Fuzzy Logic (<https://www.geeksforgeeks.org/fuzzy-logic-introduction/>)

Figure 4 – Fuzzy Logic architecture (<https://www.geeksforgeeks.org/fuzzylogic-introduction/>)

Figure 5 – Fuzzy Logic model recommends for every movie genre

Figure 6 – Results for Content based model recommendation using movie title and genre

Figure 7 – Fuzzy Logic Evaluation: Change in Threshold will change the overall results

Figure 8 – Content-Based ML model Evaluation: Change in Threshold will change the overall results

9 List of tables

Table 1 - Movie Recommendation

Table 2 - Fuzzification and Defuzzification ((Katiyar, 2022))

